

README

Part 2: 从 Bigram 到 MLP — 多层感知机名字生成

> 从只看 1 个字符，升级到看 3 个字符的上下文窗口！

章节导航

序号	章节	内容
01	[从 Bigram 到 MLP](01_introduction.md)	数据集准备、block_size、Train/Dev/Test 划分
02	[MLP 架构](02_mlp_architecture.md)	Embedding 层、前向传播、CrossEntropy Loss
03	[训练与评估](03_training_and_eval.md)	Minibatch SGD、学习率、过拟合诊断、采样生成

学习路线图

Part 1 (Bigram)

|
| "Bigram 只看 1 个字符，太少了！"



| Part 2: MLP |

| |

| ① block_size=3 的数据集 | ———→ 01_introduction.md

| ② Embedding + MLP 架构 | ———→ 02_mlp_architecture.md

| ③ 训练、评估、采样 | ———→ 03_training_and_eval.md

| |

|
| "训练起来了，但不稳定..."



Part 3 (优化器与初始化)

学完这一部分你能...

- 理解 Embedding：把离散的字符变成连续向量
- 搭建一个 两层 MLP (Embedding → 隐藏层 → 输出层)
- 掌握 Train/Dev/Test 数据划分的正确姿势
- 用 Minibatch SGD 高效训练
- 诊断 过拟合，看懂 train loss vs dev loss
- 从训练好的模型 采样生成 新名字

课后作业

完成教程后，去这里做练习：

[Assignment 2](../../assignments/assignment_2/)

相关资源

- Andrej Karpathy 原视频：[Building makemore Part 2: MLP](https://www.youtube.com/watch?v=TCH_1BHY58I)
- Bengio et al. 2003 论文：[A Neural Probabilistic Language Model](https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf)

[← 上一章：Part 1 Bigrams](../Part1_bigrams/tutorial/README.md) | [下一章：Part 3 BatchNorm →](../Part3_batchnorm/tutorial/README.md)

01_introduction

01 — 从 Bigram 到 MLP

前置知识：Bigram 的局限

在 [Part 1](../Part1_bigrams/) 里，我们做了一个 Bigram 语言模型：只看前 1 个字符，预测下一个字符。

比如名字 **emma**，Bigram 看到的是：

```
,
. → e
e → m
m → m
m → a
a → .
,
```

△ 问题来了：当你看到 **m** 的时候，模型完全不知道前面是 **e** 还是别的什么字母。它只知道 "m 后面可能跟什么"，但不知道 "前面是 e 的 m 后面应该跟什么"。

这就像你只看了一个字的上下文就要猜下一个字 —— 信息太少了！

> 语言模型的核心思想：上下文越长，预测越准。

这一课能学到什么

学完 Part 2，你将：

1. 把上下文从 1 个字符扩展到 3 个字符 (block_size=3)

2. 理解 Embedding —— 把字符变成向量
3. 搭建一个 多层感知机 (MLP) , 比 Bigram 强得多
4. 学会 Train/Dev/Test 数据划分
5. 用 Minibatch SGD 高效训练

数据集准备：block_size=3

为什么看 3 个字符？

Bengio 等人 2003 年的论文中提出：用前 3 个字符预测下一个字符，效果比只看 1 个好很多。

> block_size (也叫 context length) : 用多少个历史字符来预测下一个。

展开数据集

当 block_size=3 时, 名字 emma 展开成：

,

block_size=3 时, 名字 "emma" 展开为：

输入 → 输出(预测)

... → e

..e → m

.em → m

emm → a

mma → .(结束)

,

你看，每一行的输入都是 3 个字符（不够的用 . 填充），输出是下一个字符。

对应到代码：

```
`python
block_size = 3

for word in words[:5]: # 先看前 5 个名字
    context = [0] * block_size # [0, 0, 0] → 0 代表 '.'
    for ch in word + '.':
        ix = stoi[ch]
        X.append(context)
        y.append(ix)
        context = context[1:] + [ix] # 滑动窗口！
```

,

注意 `context = context[1:] + [ix]` —— 这就是滑动窗口，每次把最老的字符丢掉，加入新字符。

> 完整代码见 `../scripts/01_explore_data.py` (`../scripts/01_explore_data.py`) 和 `../scripts/02_dataset_with_context.py` (`../scripts/02_dataset_with_context.py`)

数据长什么样？

```
python
```

$X.shape = (N, 3) \rightarrow N$ 是样本数, 3 是 `block_size`

$y.shape = (N,)$ $\rightarrow$ 每个样本的目标字符索引

```
print(X[:5])
```

```
tensor([[0, 0, 0], # ...  $\rightarrow$  e
```

```
[0, 0, 5], # ..e  $\rightarrow$  m
```

```
[0, 5, 13], # .em  $\rightarrow$  m
```

```
[5, 13, 13], # emm  $\rightarrow$  a
```

```
[13, 13, 1])) # mma  $\rightarrow$  .
```

```
print(y[:5])
```

```
tensor([ 5, 13, 13, 1, 0])
```

```
,
```

每个数字是字符的索引 (0='.', 1='a', 2='b', ...)。

✂ Train / Dev / Test 划分

我们把所有名字分成三份：

```
,
```

全部名字数据 (32033 个名字)			
Train (80%)	Dev(10%)	Test (10%)	
~25626 个	~3203 个	~3204 个	
用来训练模型	调超参数	最终评估 (只用一次)	
(反复使用)	(适量用)	(神圣不可侵犯)	

```
,
```

为什么需要三份？

数据集	用途	使用频率
Train	训练模型参数（权重）	每个迭代都用
Dev （验证集）	调超参数（学习率、embedding 维度等）	定期评估
Test	最终报告模型效果	**只用一次**

△ 如果你用 Test 集来调参数，那就等于 "考试前看到了答案" —— 你的模型看起来很好，但遇到新数据就不行了。

```
python
import random
random.seed(42)
random.shuffle(words)

n1 = int(0.8 * len(words))
n2 = int(0.9 * len(words))

Xtr, Ytr = build_dataset(words[:n1]) # Train
Xdev, Ydev = build_dataset(words[n1:n2]) # Dev
Xte, Yte = build_dataset(words[n2:]) # Test
```

下一步

数据准备好了，接下来该搭建模型了！

[02 — MLP 架构](02_mlp_architecture.md)

02_mlp_architecture

02 — MLP 架构

Embedding 层：把字符变成向量

什么是 Embedding？

在 Bigram 里，我们用 one-hot 向量表示字符 —— 一个长度 27 的向量，只有一个位置是 1。这样太浪费了，而且字符之间没有任何关系。

Embedding 的想法：给每个字符分配一个低维向量（比如 2 维或 10 维），让模型自己学习这些向量应该长什么样。

```
字符 'a' (索引1) → C[1] → [0.3, -0.1] (2维向量)
字符 'm' (索引13) → C[13] → [-0.5, 0.8]
字符 '!' (索引0) → C[0] → [0.1, 0.2]
```

C 是一个形状为 $(27, 2)$ 的矩阵，27 个字符，每个字符 2 维。这 2 维就是字符在“空间”中的位置。

代码实现

```
python
import torch
```

Embedding 矩阵：27 个字符，每个用 2 维向量表示

```
C = torch.randn((27, 2))
```

查表：输入索引 5，得到对应的 2 维向量

```
print(C[5]) # tensor([0.xx, -0.xx])
print(C[[5, 13]]) # 也可以一次查多个  $\rightarrow (2, 2)$  矩阵
```

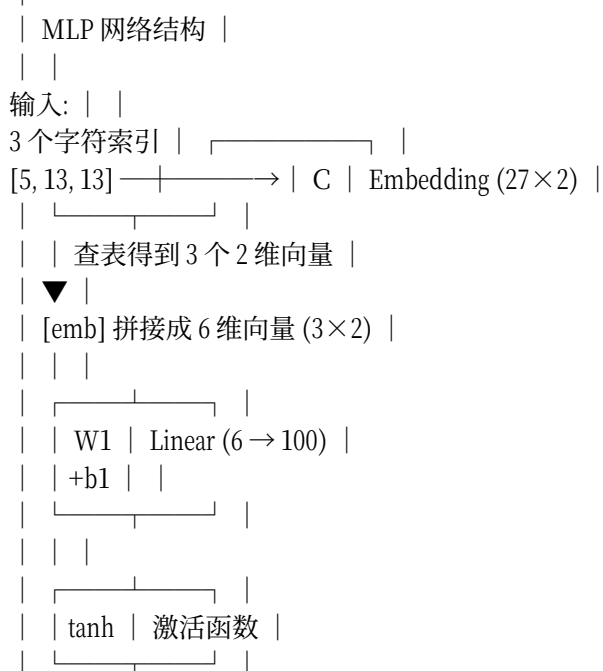
关键点： C 就是我们要训练的参数！一开始是随机的，训练过程中会慢慢变成有意义的向量。

> 完整代码见 [\[../scripts/03_embedding.py\]\(../scripts/03_embedding.py\)](#)

MLP 前向传播

网络结构

我们参考 Bengio 2003 论文的架构，搭一个两层 MLP：





一步步写代码

Step 1: Embedding + 拼接

```
python
```

X 是 $(N, 3)$ 的索引矩阵， C 是 $(27, 2)$ 的 Embedding

```
emb = C[X] # (N, 3, 2) → 每个样本 3 个字符，每个字符 2 维
emb_cat = emb.view(emb.shape[0], -1) # (N, 6) → 拼成一维向量
```

$C[X]$ 高级索引详解

$C[X]$ 是 PyTorch 的高级索引操作，也是 Part 2 最核心的操作之一。让我们拆开看：

```
python
```

```
C = torch.randn(27, 2) # 27 个字符，每个字符用 2 维向量表示
X = torch.tensor([[0, 0, 0], # 第 1 个样本：上下文是 "...", 即 "...", "...",
[0, 0, 5]]) # 第 2 个样本：上下文是 "..e", 即 "...", "...", "e"
emb = C[X] # 形状：(2, 3, 2)
```

发生了什么？

X 中的每个整数都被当作 C 的行索引，替换成 C 中对应的行：

```
X[0] = [0, 0, 0] → C[[0, 0, 0]] = [C[0], C[0], C[0]] → 3 个 2 维向量
X[1] = [0, 0, 5] → C[[0, 0, 5]] = [C[0], C[0], C[5]] → 3 个 2 维向量
```

形状推导规则：

```
C 的形状：(27, 2) ← 词汇表大小 × 嵌入维度
X 的形状：(N, 3) ← N 个样本 × 上下文长度
C[X] 的形状：(N, 3, 2) ← N 个样本 × 3 个字符 × 2 维向量
```

> 和 one-hot 的关系： $C[X]$ 等价于先 $F.one_hot(X, 27) @ C$ ，但高效得多——one-hot 需要创建 $(N, 3, 27)$ 的稀疏矩阵再做矩阵乘法，而 $C[X]$ 直接查表，零拷贝。

view(-1) 详解

```
`python
emb = C[X] # (N, 3, 2)
emb_cat = emb.view(emb.shape[0], -1) # (N, 6)
```

view 是什么？

view 改变 tensor 的形状，但不改变数据在内存中的排列。它只是改变了 tensor 的"视图" (shape 和 stride)。

-1 是什么意思？

-1 告诉 PyTorch："这个维度你帮我自动算"。计算规则是：

```
`
总元素数 =  $N \times 3 \times 2 = 6N$ 
view(N, -1)  $\rightarrow$  第 2 维 =  $6N / N = 6$ 
所以结果是 (N, 6)
```

具体例子：

```
`python
emb.shape = (32, 3, 2) # 32 个样本，每个有 3 个 2 维向量
emb.view(32, -1)  $\rightarrow$  (32, 6) # 3 个 2 维向量拍扁成 1 个 6 维向量
emb.view(32, 6)  $\rightarrow$  (32, 6) # 等价写法，但 -1 更灵活（不用手动算 6）
emb.view(-1)  $\rightarrow$  (64,) # -1 也可以用在第 1 维，完全展平
emb.view(32, 3, 2)  $\rightarrow$  (32, 3, 2) # 形状不变
```

> $\triangle$ **view** vs **reshape**：**view** 要求内存连续 (contiguous)，如果 tensor 不连续会报错。**reshape** 更灵活，可能返回拷贝。初学阶段两者可以互换，但 **view** 更高效。

emb.view(emb.shape[0], -1) 就是把 3 个 2 维向量拍扁成 1 个 6 维向量。**-1** 让 PyTorch 自动算第二维。

Step 2: 隐藏层

```
`python
W1 = torch.randn((6, 100)) # 6  $\rightarrow$  100
b1 = torch.randn(100)
h = torch.tanh(emb_cat @ W1 + b1) # (N, 100)
```

tanh 把输出压到 $[-1, 1]$ 之间，给网络非线性能力。

Step 3: 输出层

```
`python
W2 = torch.randn((100, 27)) # 100  $\rightarrow$  27 (27 个字符)
b2 = torch.randn(27)
logits = h @ W2 + b2 # (N, 27)
```

Step 4: Softmax $\rightarrow$ 概率

```
`python
counts = logits.exp() # 等价于 Bigram 里的 N 矩阵
```



```

prob = counts / counts.sum(1, keepdim=True) # 归一化
`
> 完整代码见 [../scripts/04_mlp_forward.py](../scripts/04_mlp_forward.py)
>
> 训练过程中 loss 曲线可视化：
>
> `python
> # 可视化：Loss 曲线 → 生成 ../images/cell028_output01.png
> import matplotlib.pyplot as plt
>
> plt.plot(stepi, lossi)
> plt.xlabel('Step')
> plt.ylabel('Loss')
> plt.title('Training Loss')
> plt.savefig('../images/cell028_output01.png', dpi=150, bbox_inches='tight')
> plt.show()
> `
>
> ![训练 Loss 曲线](../images/cell028_output01.png)

```

CrossEntropy Loss

为什么不用手动算？

你可以手动算 loss：

```
`python
```

手动版本（不推荐）

```

logits = h @ W2 + b2
counts = logits.exp()
prob = counts / counts.sum(1, keepdim=True)
loss = -prob[torch.arange(N), Y].log().mean()
`

```

⚠ 但这样有三个问题：

1. 数值不稳定：`exp()` 对大数会溢出 → `inf`
2. 效率低：PyTorch 没法对这三步做融合优化
3. 反向传播复杂：自己写 grad 容易出错

推荐做法

```

`python
import torch.nn.functional as F

logits = h @ W2 + b2
loss = F.cross_entropy(logits, Y) # 一步到位！

```

`F.cross_entropy` 内部做了：

1. 先减去最大值（防止溢出）
2. 用 log-sum-exp 技巧算 log-softmax
3. 取负对数似然的均值

数学等价，但更快、更稳定。

课后练习

Q1: Embedding 维度

> 如果把 Embedding 从 2 维改成 10 维，`W1` 的形状应该是什么？（`block_size=3`）

<details>

<summary>点击查看答案</summary>

`W1` 的输入维度 = `block_size` × `emb_dim` = 3 × 10 = 30。所以 `W1` 的形状是 `(30, 100)`。

</details>

Q2: view vs reshape

> `emb.view(N, -1)` 和 `emb.reshape(N, -1)` 有什么区别？什么情况下用哪个？

<details>

<summary>点击查看答案</summary>

- `view` 要求张量在内存中是连续的（contiguous），更快但不总是能用
- `reshape` 任何情况都能用，如果内存不连续会自动复制一份
- 实践中：先试 `view`，报错了再用 `reshape`

</details>

Q3: 为什么用 tanh？

> 隐藏层为什么用 `tanh` 而不是 `sigmoid` 或 `ReLU`？你觉得各有什么优缺点？

<details>

<summary>点击查看答案</summary>

- `tanh`：输出 `[-1, 1]`，零中心，梯度比 `sigmoid` 大 → 训练更快
- `sigmoid`：输出 `[0, 1]`，两端梯度极小 → 容易梯度消失
- `ReLU`：简单高效，但 Bengio 2003 论文用的是 `tanh`，这里是跟原论文一致

实际上在后续 Part 中会看到，`tanh` 在这里效果不错，但隐藏层太大会导致 "dead neurons" 问题。

</details>

下一步

架构搭好了，接下来训练它！

[03 — 训练与评估](03_training_and_eval.md)

03_training_and_eval

03 — 训练与评估

Minibatch SGD

为什么要用小批量？

全部数据有 22 万+ 个样本。如果每个迭代都过一遍所有数据：

- 一个 epoch 要算 228146 次前向 + 反向传播
- GPU 利用率可能很低（矩阵太小）
- 梯度虽然精确，但更新太慢

Minibatch 的想法：每次随机抽一小批数据（比如 32 或 64 个样本），用这批数据算梯度，更新参数。

全量梯度下降：Minibatch SGD：

用全部 22 万样本	随机抽 32 个样本
算一次精确梯度	算一次近似梯度
更新一次参数	更新一次参数
很慢但很准	很快但有点噪声
1 步 / 几秒	1000 步 / 几秒

△ Minibatch 的梯度有噪声（不完全精确），但实践证明这个噪声反而有助于逃离局部最优。

代码实现

```
python
for i in range(1000):
```

随机选 32 个样本

```
ix = torch.randint(0, Xtr.shape[0], (32,))
```

前向传播（只用这 32 个）

```
emb = C[Xtr[ix]] # (32, 3, 2)
h = torch.tanh(emb.view(-1, 6) @ W1 + b1) # (32, 100)
```

```
logits = h @ W2 + b2 # (32, 27)
loss = F.cross_entropy(logits, Ytr[ix])
```

反向传播

```
for p in parameters:
    p.grad = None
loss.backward()
```

更新参数

```
for p in parameters:
    p.data += -lr * p.grad
```

注意：`Xtr[ix]` 和 `Ytr[ix]` 只取了 32 个样本，所以前向传播和反向传播都很快。

> 完整代码见 `./scripts/05_minibatch_training.py` (`./scripts/05_minibatch_training.py`)

学习率调度

如何选择学习率？

Andrej 在视频里演示了一个技巧：学习率搜索。

```
python
```

试 1000 个不同的学习率，从 0.001 到 1

```
lre = torch.linspace(-3, 0, 1000) # 指数空间
lrs = 10 ** lre # 0.001 到 1.0
```

```
lri = []
lossi = []
```

```
for i in range(1000):
    ix = torch.randint(0, Xtr.shape[0], (32,))

    emb = C[Xtr[ix]]
    h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
    logits = h @ W2 + b2
    loss = F.cross_entropy(logits, Ytr[ix])
```

```
for p in parameters:
    p.grad = None
loss.backward()
```

```
lr = lrs[i]
for p in parameters:
    p.data += -lr * p.grad

lri.append(lre[i])
lossi.append(loss.item())
```

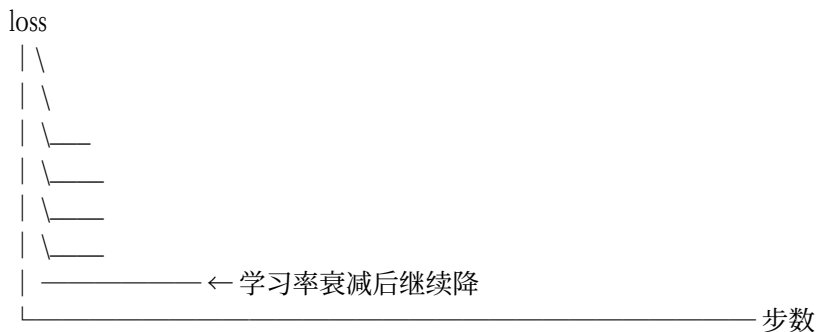
画图：loss vs 学习率的指数

```
plt.plot(lri, lossi)
```

你会看到 loss 先下降，然后在一个点之后开始爆炸 —— 那个最低点附近就是好学习率。

学习率衰减

找到好的学习率后（比如 0.1），训练到 loss 趋于平稳，再把学习率缩小（比如降到 0.01），继续训练。这就是学习率衰减：



过拟合诊断

Train Loss vs Dev Loss

训练过程中，我们要同时看 train loss 和 dev loss：

```
python
@torch.no_grad() # 不算梯度，节省内存
def evaluate(X, Y):
    emb = C[X]
    h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
    logits = h @ W2 + b2
    loss = F.cross_entropy(logits, Y)
    return loss.item()

print(f"Train loss: {evaluate(Xtr, Ytr):.4f}")
print(f"Dev loss: {evaluate(Xdev, Ydev):.4f}")
```

三种情况：

情况 1: 欠拟合
Train loss: 2.5 ← 都很高
Dev loss: 2.6 ← 差距小
→ 模型太小 / 训练不够

情况 2: 刚刚好

Train loss: 2.1

Dev loss: 2.2

→ 差距小, 数值低

情况 3: 过拟合

Train loss: 1.5 ← 很低

Dev loss: 2.5 ← 高很多

→ 模型 memorize 了训练集

△ 我们这个 MLP 模型参数量 (约 3,500, 可用 `sum(p.numel() for p in parameters)` 计算) 远小于数据量 (约 22 万样本), 所以不太会过拟合。但记住这个诊断方法, 后面的模型会用上。

Embedding 可视化

训练完之后, C 矩阵 (27×2) 变成了什么样?

```
`python
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 8))
plt.scatter(C[:, 0].data, C[:, 1].data, s=200)
for i in range(C.shape[0]):
    plt.text(C[i, 0].item(), C[i, 1].item(), itos[i],
             ha="center", va="center", color="white")
plt.grid("minor")
```

你会发现有趣的模式:

- 元音字母 (a, e, i, o, u) 聚在一起 → 模型学到它们功能相似
- 相似功能的辅音 (如 b/p/d/t) 也靠得很近
- . (起始/结束符) 在比较远的位置

这就是 Embedding 的魔力: 模型自己学会了字符之间的相似关系!

> 完整代码见 `../scripts/06_visualize_embedding.py` (`../scripts/06_visualize_embedding.py`)

>

> Embedding 可视化代码:

>

> `python

> # 可视化: Embedding 2D 投影 → 生成 `../images/cell031_output00.png`

> import matplotlib.pyplot as plt

>

> plt.figure(figsize=(8, 8))

> plt.scatter(C[:, 0].data, C[:, 1].data, s=200)

> for i in range(C.shape[0]):

> plt.text(C[i, 0].item(), C[i, 1].item(), itos[i],

> ha="center", va="center", color='white')

> plt.grid('minor')

> plt.title('Embedding Space (2D)')

```
> plt.savefig('../images/cell031_output00.png', dpi=150, bbox_inches='tight')
> plt.show()
> `
>
>
> ![Embedding 可视化](../images/cell031_output00.png)
```

采样生成

训练好了，让模型生成新名字！

```
`python
g = torch.Generator().manual_seed(2147483647)

for _ in range(20):
    out = []
    context = [0] * block_size # [0, 0, 0] → 开始

    while True:
        emb = C[torch.tensor([context])] # (1, 3, 2)
        h = torch.tanh(emb.view(1, -1) @ W1 + b1)
        logits = h @ W2 + b2
        prob = F.softmax(logits, dim=1)
```

从概率分布中采样

```
ix = torch.multinomial(prob, num_samples=1, generator=g).item()
context = context[1:] + [ix]

if ix == 0: # 遇到结束符
    break
out.append(itos[ix])

print(''.join(out))
`
```

输出大概是：

```
`
mora
kiah
mel
...
`
```

比 Bigram 好不少！虽然还是有些奇怪的名字，但至少更像真正的英文名了。

> 完整代码见 [[../scripts/07_sampling.py](#)](../scripts/07_sampling.py)

课后作业

完成教程后，去做练习巩固：

课后练习

Q1: Batch Size 选择

> batch_size=32 和 batch_size=1024 各有什么优缺点？如果 GPU 内存够大，应该选哪个？

<details>

<summary>点击查看答案</summary>

- 小 batch (32)：梯度噪声大 → 探索性强，但训练曲线不平滑
- 大 batch (1024)：梯度更精确 → 训练更稳，但每次更新慢，可能陷入局部最优
- 实践中：根据 GPU 内存选尽可能大的 batch，配合适当的学习率调整。常见范围 32~256。

</details>

Q2: Embedding 维度的影响

> 如果把 Embedding 维度从 2 增加到 10，模型效果会变好吗？参数量会增加多少？

<details>

<summary>点击查看答案</summary>

- Embedding 参数： $27 \times 2 = 54 \rightarrow 27 \times 10 = 270$ (增加 216)
- W1 参数： $6 \times 100 = 600 \rightarrow 30 \times 100 = 3000$ (增加 2400)
- 总参数增加约 2600 个
- 效果：通常维度更大表达能力更强，但如果数据量不够，可能会过拟合
- 2 维只是为了方便可视化，实际应用中通常用更高维度

</details>

Q3: Dev Loss 停滞

> 训练到 train loss = 2.1, dev loss = 2.3，继续训练 train loss 还在降但 dev loss 不降了。这说明什么？该怎么办？

<details>

<summary>点击查看答案</summary>

这说明模型在训练集上过拟合了 —— 它在 memorize 训练数据而不是学习通用规律。

可能的解决方案：

1. 增大模型（增加隐藏层大小）—— 如果 train loss 还能降，说明模型容量不够
2. 增加正则化（dropout, weight decay）
3. 增加数据量
4. 在这个特定案例中，模型参数只有 ~3,500，数据有 ~22 万，所以更可能是模型太小了（欠拟合），而不是过拟合

</details>

下一课预告

这一课我们搭了 MLP，训练起来了，效果比 Bigram 好。但如果你仔细观察训练过程，会发现一些问题：

- 训练初期 loss 下降很快，但后面越来越慢
- 不同层的梯度大小差异很大
- tanh 的输出很多集中在 -1 和 1 附近（"饱和"了）

Part 3 将解决这些问题：引入 BatchNorm、讨论初始化策略、理解梯度流。

想提前预习？看 Andrej 的 [Building makemore Part 3: Activations & Gradients](<https://www.youtube.com/watch?v=P6sfmUTpUmc>)